

Prompts Are Just Context

What the model really sees when you chat · video 2:06

This lesson answers one question: what does a language model actually receive when you chat with it? Not speech bubbles, and not a memory of you, but **one document of tokens**, built by a **chat template**. The app **resends the whole conversation** on every turn, so everything the model knows right now is in that **context**. Prompting is **writing the beginning of a document** whose likely continuation is the answer you want.

How to use this guide: master one concept before moving to the next.

1. Watch the whole video once, without stopping.
2. Then take the concepts **in order**. For each one: read the short explanation, look at the picture from the video (▶ is the time in the video), and answer every question in its *Check* without looking anything up.
3. Check your answers in the **Answers** section at the end (page 9).
4. **Pass mark: every question in the check correct.** If you missed one, rewatch that concept's video segment, reread the explanation, and redo only the questions you missed. Tick the concept off below, then move on.

It is strongly suggested that you **master each concept before starting the next one**. Each concept builds on the previous ones.

Concepts in this lesson

CONCEPT	VIDEO SEGMENT	QUESTIONS	MASTERED
1. A chat is one document	▶ 0:08-0:49	4	☐
2. Tokens are the model's whole world	▶ 0:49-0:58	3	☐
3. No memory: the whole chat is sent again	▶ 0:58-1:15	3	☐
4. Everything is context	▶ 1:15-1:41	4	☐
5. Prompting is writing the start of a document	▶ 1:41-2:03	3	☐

Before you start: this series builds on How LLMs Work, especially episode 1 (the generation loop), episode 2 (tokens) and episode 14 (chat templates and fine-tuning). The code uses Python and the transformers library; see `code/p01_prompt_is_context` in the repository.

CONCEPT 1 OF 5

A chat is one document

▶ Video 0:08–0:49

A chat app shows **speech bubbles**, turns and what looks like a memory. The model never sees any of that. Before each request, the app uses the model's **chat template** to flatten the system message, your messages and the assistant's earlier replies into **one document**. **Special tokens** mark who is speaking; for Qwen2.5 they are `<|im_start|>` and `<|im_end|>`, and each model family has its own.

▶ 0:34 The four messages, flattened by Qwen2.5's real chat template. Orange: special tokens that mark who is speaking.

▶ 0:47 The document ends with an open assistant turn, so the most natural continuation is the reply (illustrative reply).

Look at the very end: the document stops right after a new `<|im_start|>assistant` marker. The most natural way to continue this document is the assistant's reply. **Answering is continuing the text**, with the same next-token loop as always.

KEY IDEA

A conversation reaches the model as **one document**, built by a chat template, ending with an **open assistant turn** that the model continues.

Check 1: A chat is one document

Answer all 4 questions before looking at the answers. Pass mark: all correct.

1.1 MULTIPLE CHOICE

What does the model receive when you send the fourth message of a chat?

- A. Only your newest message
- B. Your newest message plus a summary the model wrote earlier
- C. One document containing the whole conversation, built by a chat template
- D. A list of speech bubbles, each processed separately

1.2 SHORT ANSWER

Why does the templated document end with `<|im_start|>assistant` and nothing after it?

.....

1.3 TRUE OR FALSE

"Every model family uses the same special tokens, `<|im_start|>` and `<|im_end|>`."

True ☐ False ☐

1.4 MULTIPLE CHOICE

A user sends one message and no system message. Qwen2.5's template still produces a system turn. What does it contain?

- A.** Nothing: an empty system turn
- B.** “You are Qwen, created by Alibaba Cloud. You are a helpful assistant.”
- C.** The user's message, repeated
- D.** The name of the app

CONCEPT 2 OF 5

Tokens are the model's whole world

▶ Video 0:49-0:58

Then the document becomes **tokens** (How LLMs Work, episode 2). The four-message chat is **55 tokens**, and each one is just a number: `<|im_start|>` is 151644, `system` is 8948, a line break is 198. Special tokens are tokens like any other, with their own IDs.



▶ 0:56 The first 12 of the 55 real tokens, with their IDs. Orange: special tokens.

KEY IDEA

That list of numbers is the model's **entire world**: anything that is not in it does not exist for the model.

Check 2: Tokens are the model's whole world

Answer all 3 questions before looking at the answers. Pass mark: all correct.

2.1 CALCULATION

The templated four-message chat is 55 tokens. Its first user turn alone (with the system message) is 28 tokens. How many tokens do the assistant's reply and the follow-up question add, including their template tokens?

2.2 TRUE OR FALSE

"`<|im_end|>` is split into several ordinary tokens such as `<`, `|` and `im.`"

True ☐ False ☐

2.3 SHORT ANSWER

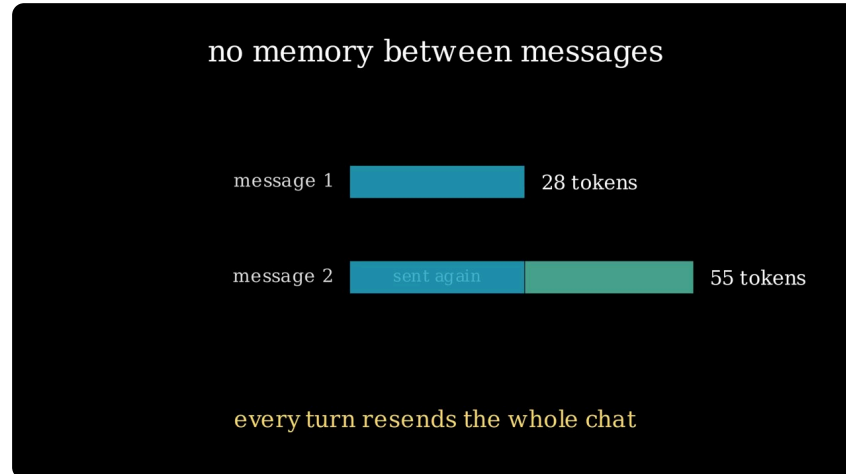
Your question "*How long should I boil an egg?*" is 8 tokens on its own. Why is the first request bigger (28 tokens)?

CONCEPT 3 OF 5

No memory: the whole chat is sent again

▶ Video 0:58-1:15

The model has **no memory between messages**. Every time you send one, the app sends the **whole conversation again, from the top**. In our chat, the first question is a 28-token request; the follow-up is a 55-token request that contains the first one again.



▶ 1:14 The second request contains the first one again: 28 tokens, then 55.

So a chat gets longer to read on every turn: a third exchange would make the request 79 tokens. This is why long chats get slower and cost more, and why they eventually hit a limit (next episode).

KEY IDEA

The “memory” of a chat is the app **resending the transcript**. The model itself starts from scratch on every request.

Check 3: No memory: the whole chat is sent again

Answer all 3 questions before looking at the answers. Pass mark: all correct.

3.1 CALCULATION

The requests on three turns are 28, 55 and 79 tokens. How many tokens does the model read in total over the three requests?

3.2 TRUE OR FALSE

“After you tell an assistant your name in message 1, the model stores it in its weights for message 2.”

True ☐ False ☐

3.3 MULTIPLE CHOICE

An app deletes the oldest messages of a long chat to save money. What happens?

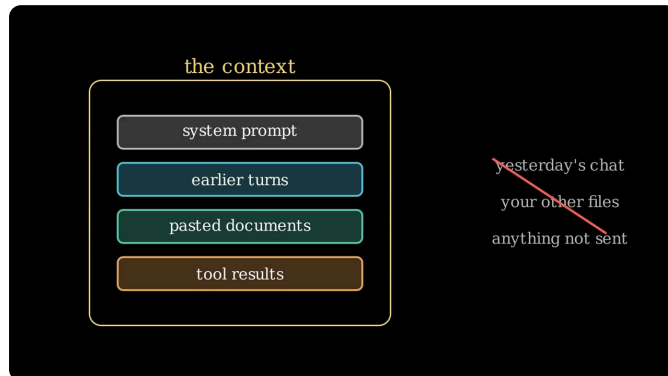
- A. Nothing: the model remembers them anyway
- B. The model can no longer use what they said
- C. The model's weights are reset
- D. The chat template stops working

CONCEPT 4 OF 5

Everything is context

▶ Video 1:15-1:41

Everything the model knows about you, right now, lives in that text: the **system prompt**, **earlier turns**, **documents you pasted** and **results from tools** (episode 7). Yesterday's chat or a file you did not send are not there, so the model cannot use them.



▶ 1:27 Everything the model knows about you right now is in the context.

```
1 tok = AutoTokenizer.from_pretrained("Qwen/Qwen2.5-0.5B-Instruct")
2 messages = [{"role": "system", "content": "You are a friendly ..."},
3             {"role": "user", "content": "How long should I boil an egg?"}, ...]
4 text = tok.apply_chat_template(messages, tokenize=False, add_generation_prompt=True)
5
6 ids = tok(text)["input_ids"]
7 print(len(ids)) # 55
```

code/p01_prompt_is_context · link below

▶ 1:40 The episode's code: messages → chat template → tokens → count.

In code it is three steps: a list of messages, `apply_chat_template` to get one string, then tokenize and count. Run `code/p01_prompt_is_context/what_the_model_sees.py` to see the full document, the token IDs and the size of each request.

KEY IDEA

If it isn't in the context, the model can't see it. Apps that seem to “remember” or “search” are putting text into the context for you.

Check 4: Everything is context

Answer all 4 questions before looking at the answers. Pass mark: all correct.

4.1 MULTIPLE CHOICE

Which of these can the model use when answering your next message?

- A. A PDF you pasted earlier in this chat
- B. A chat you had with it yesterday, in another window
- C. A file on your computer you never uploaded
- D. Your browser history

4.2 PUT IN ORDER

Put in order what happens to your message: *tokens · chat template · the model continues · list of messages*.

4.3 SHORT ANSWER

An assistant answers a question about today's news correctly, although its training data is old. Using this lesson, how is that possible?

4.4 HANDS-ON CODE

Try it yourself. Run the code below. (a) What system prompt appears, although you did not write one? (b) How many tokens is the request? (c) Add a system message “*You are a friendly cooking assistant.*” at the start of messages : how many tokens now?

```
from transformers import AutoTokenizer

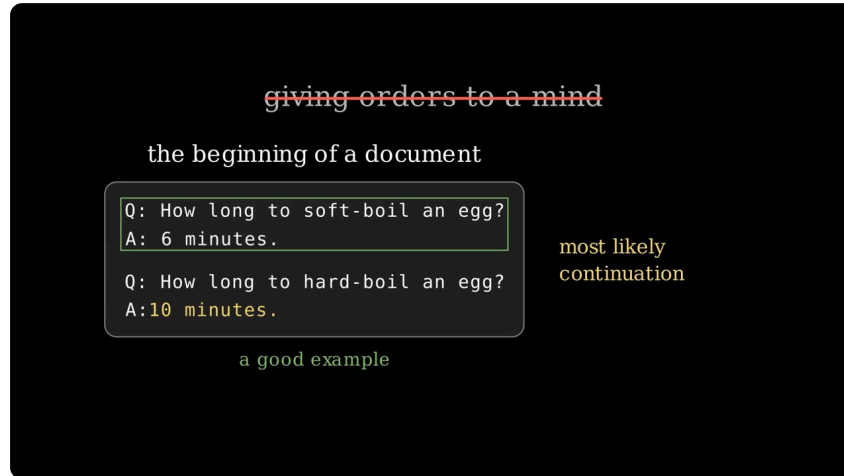
tok = AutoTokenizer.from_pretrained("Qwen/Qwen2.5-0.5B-Instruct")
messages = [{"role": "user", "content": "How long should I boil an egg?"}]
text = tok.apply_chat_template(messages, tokenize=False, add_generation_prompt=True)
print(text)
print(len(tok(text)["input_ids"]))
```

CONCEPT 5 OF 5

Prompting is writing the start of a document

▶ Video 1:41-2:03

This is why prompting works the way it does. You are not giving orders to a mind. You are **writing the beginning of a document**, so that the answer you want is its **most likely continuation**.



▶ 1:56 A prompt is the beginning of a document; a good example makes the continuation easy to predict.

Clear instructions and a **few good examples** (called *few-shot* prompting) make that continuation easy to predict: after a soft-boiled question answered in the format “A: 6 minutes.”, the likely continuation of the hard-boiled question is an answer in the same format.

KEY IDEA

A good prompt sets up a document whose natural next words are the answer you want, in the form you want.

Check 5: Prompting is writing the start of a document

Answer all 3 questions before looking at the answers. Pass mark: all correct.

5.1 MULTIPLE CHOICE

You want answers as a single number of minutes. Which prompt makes that most likely?

- A. “Eggs?”
- B. “Tell me about boiling eggs.”
- C. “Q: Soft-boiled egg? A: 6\nQ: Hard-boiled egg? A:”
- D. “Please be smart.”

5.2 TRUE OR FALSE

“Few-shot prompting means retraining the model on a few examples.”

True ☐ False ☐

5.3 SHORT ANSWER

Rewrite “Translate: cat” as the start of a document whose most likely continuation is the Italian word.

.....

ANSWERS

Answers and explanations

Mark each answer right or wrong. If a concept has any wrong answer, rewatch its segment (▶) and redo the questions you missed before moving on.

1. A chat is one document · rewatch ▶ 0:08-0:49

1.1 C. The app flattens every message into one templated document.

1.2 So that the most likely continuation is the assistant's reply. The model only ever continues text; opening the assistant's turn tells it what to write next.

1.3 False. Templates differ between model families; that is why the code calls the model's own `apply_chat_template`.

1.4 B. This template inserts a default system prompt when none is given (check it with the hands-on code in concept 4). Other models do not.

2. Tokens are the model's whole world · rewatch ▶ 0:49-0:58

2.1 27. $55 - 28 = 27$.

2.2 False. It is a single special token, ID 151645 in Qwen2.5.

2.3 The system message and the template's special tokens and role names are added. Everything in the document counts: roles, markers, line breaks and the system prompt.

3. No memory: the whole chat is sent again · rewatch ▶ 0:58-1:15

3.1 162. $28 + 55 + 79 = 162$: each request is read in full.

3.2 False. The weights do not change while chatting. Your name is known in message 2 only because the app resends message 1.

3.3 B. If it is not in the context, the model cannot see it.

4. Everything is context · rewatch ▶ 1:15-1:41

4.1 A. Only the pasted PDF is part of the context that is sent.

4.2 list of messages → chat template → tokens → the model continues. The messages are flattened into one document, tokenized, then continued.

4.3 A tool (such as web search) put today's news into the context. Tool results are text added to the context, like anything else.

4.4 (a) "You are Qwen, created by Alibaba Cloud. You are a helpful assistant." (b) 37. (c) 28. Qwen2.5's template inserts a default system prompt when none is given. Your shorter system message replaces it, so the request shrinks from 37 to 28 tokens.

5. Prompting is writing the start of a document · rewatch ▶ 1:41-2:03

5.1 C. The example sets the format, so the likely continuation is a single number.

5.2 False. The examples are only text in the context; the weights do not change.

5.3 For example: "English: dog → Italian: cane\nEnglish: cat → Italian:" An example sets the pattern, and the open slot at the end is what the model continues.